7 Local LLM Families To Replace Claude/Codex (for everyday tasks)

Agent Native

There are **7 open-source model families** you can run locally that are now delivering **real-world performance surprisingly close to commercial-grade models** on many practical tasks.

> *If you don't have a Medium membership, you can read the full article for free.*

Besides leaderboard headlines, I will walk through **local setups** (e.g., **Claude Code** and **Codex**), a **hardware tier guide** (NVIDIA GPUs and Mac setups), and the **operational gotchas, failure modes and trade-offs**: latency, memory footprint, tool use, coding reliability, and instruction-following quality.

There are two big reasons why this is happening.

First, **open-source models can now handle a meaningful share of everyday work**. For a lot of coding, writing, automation, and agentic tasks, you no longer need to default to Opus 4.6 for every step.

Second, **a local fallback helps preserve expensive credits**. If you are running agent swarms, iterating heavily, or delegating lots of sub-tasks, local models are one of the easiest ways to avoid burning through your **Opus 4.6** or **Codex 5.3** credits on work that does not actually require them.

Moreover, if you had told me a year ago that an open-source model would land within striking distance of top closed models on **SWE-bench Verified**, I would have been politely skeptical.

But that performance gap has narrowed dramatically for a set of well-scoped tasks.

Recent results from models like **MiniMax M2.5**, **GLM-5**, and **Qwen3.5 (27B dense)** show **genuine competitiveness on standardized coding evaluations**, in some cases approaching parity with smaller or mid-tier commercial options.

I want to be careful here: **benchmarks are still benchmarks**. Real-world coding performance depends on far more than a single score, including tool use, context handling, consistency over long sessions, and how well a model behaves inside an actual development workflow.

But the trendline is unmistakable.

For many tasks, open-source models are becoming the default and paid frontier models are increasingly the escalation path.

The Architecture Revolution

Traditional dense models activate every parameter for every token. A 70B model needs all 70B parameters loaded and running. MoE models are different as they have a large total parameter count but only activate a fraction (the **experts** relevant to the current task) for each token.

This is the secret sauce behind the latest Qwen 3.5 35B-A3B model, a 35B-parameter model surpasses its 235B-parameter predecessor while activating only 3B parameters per token.

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation I have been running local models for a...

agentnativedev.medium.com

This is very important because while VRAM requirements are

primarily determined by the total parameters of the model (the file size) that need to be loaded, active parameters determine the speed and compute requirements of an LLM.

This means that a 230B MoE model with 10B active params can run on more modest hardware than its dense version.

> *Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***You can read it here:** Agentic SaaS Patterns Winning in 2026, packed with real-world examples, architectures, and workflows you won't find anywhere else.*

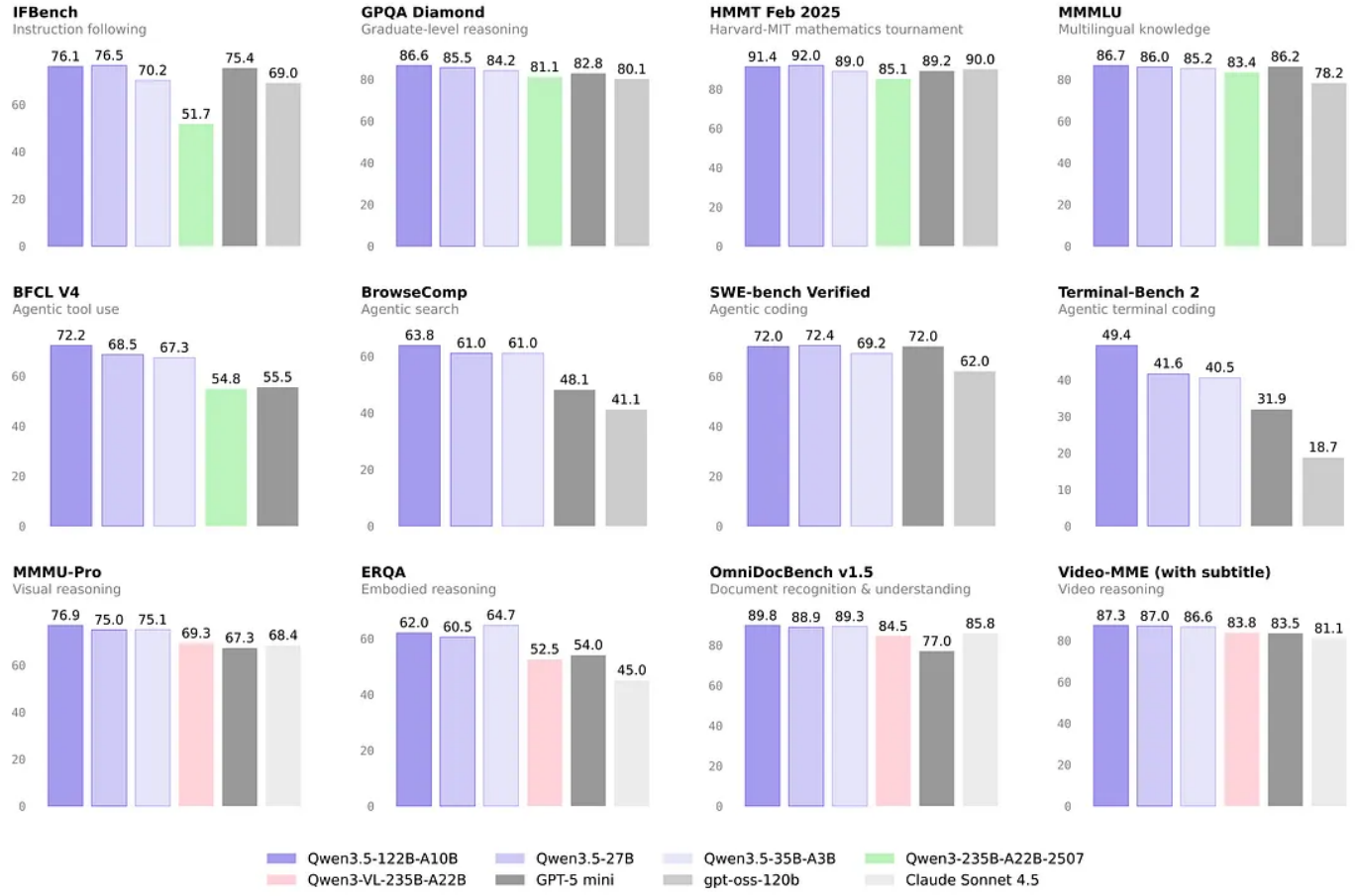
Let me break down every model worth knowing about right now.

The Qwen 3.5 Medium Series

Alibaba's Qwen team has been on an absolute tear, and the 3.5 Medium series is arguably the most important release of 2026 for local model users.

Qwen3.5–35B-A3B: This is the headliner for consumer hardware. 35 billion total parameters, but only 3 billion active at inference time thanks to its hybrid attention + MoE architecture. It surpasses the previous Qwen3–235B-A22B

flagship. It scores 69.2% on SWE-bench Verified and, critically, runs on GPUs with just 8GB of VRAM when quantized to Q4. That's a laptop GPU. That's a gaming PC from 2022.



Here's also a deep dive that I recently published on **Qwen3.5–35B-A3B:**

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for...

agentnativedev.medium.com

Qwen3.5–27B (Dense): For those with a bit more headroom, the dense 27B variant ties GPT-5 mini at 72.4% on SWE-bench Verified. Needs 16–24GB VRAM, so a single RTX 4090 or an M-series Mac handles it comfortably.

Qwen3.5–122B-A10B: The mid-range MoE option. 122B total, 10B active, scoring 72.0% on SWE-bench. Requires more memory than the 35B variant but offers better performance for those with the hardware.

Qwen3.5 Flash: The speed-optimized variant for latency-sensitive use cases. I haven't benchmarked this one extensively myself, but early reports suggest it's excellent for interactive coding where you need near-instant responses.

Qwen3-Coder Series

Separate from the Qwen 3.5 Medium series, the Qwen3-Coder family is purpose-built for coding tasks.

Qwen3-Coder-30B-A3B: Another MoE model optimized specifically for code. Needs about 18GB+ VRAM. Its Terminal-Bench 2.0 Hard score of 15.2% might look low in isolation, but Terminal-Bench Hard tests complex multi-step terminal operations that even top closed-source models struggle with.

Qwen3-Coder-480B-A35B: The flagship coder. 480B total parameters, 35B active. This is a serious model that needs serious hardware (we're talking 64GB+ Mac Unified Memory or multi-GPU setups), but it's the most capable open-source coding-specific model in the Qwen lineup.

Qwen3-Coder-Next-80B-A3B: An interesting middle ground with only 3B active parameters but 80B total. I haven't run this one locally yet, but the parameter efficiency ratio is remarkable.

But here’s what’s really interesting: it’s MIT licensed, and it was trained entirely on Huawei Ascend chips, no NVIDIA hardware involved. From a pure technical perspective, the fact that a model trained on non-NVIDIA accelerators can hit these numbers is significant.

Running GLM-5 locally requires substantial hardware. With 40B active parameters, you’re looking at 128GB+ systems. Mac Studio with 192GB unified memory, or multi-H100 setups. Not for everyone, but the MIT license means you can deploy it however you want.

Here’s a deep dive on GLM-5:

GLM-5 joins Opus and Codex: Long-horizon Agentic Tasks

GLM-5 is on par with Opus and Codex, has lowest hallucination rate and it's 6x cheaper than Opus. It also works with...

agentnativedev.medium.com

GLM-4.7

The more accessible sibling. 355B total parameters, 32B active, scoring 73.8% on SWE-bench Verified and 41% on Terminal-Bench 2.0. Also MIT licensed, also trained on Huawei Ascend. This one fits on a 64GB Mac or a dual-GPU workstation, making it a more practical choice for most developers.

I find it somewhat surreal that OpenAI has released genuinely open models under Apache 2.0.

More high-quality options for local deployment.

DeepSeek V3.2 and V3.2-Speciale

DeepSeek has been a consistent presence in the open-source model space. V3.2 scores **72–74%** on SWE-bench Verified (depending on thinking mode) and 37.7% on Terminal-Bench 2.0.

The Speciale variant offers some optimizations.

Not the highest benchmark numbers on this list, but DeepSeek models tend to punch above their weight in practical coding scenarios, at least in my experience.

Their training data and tuning makes them particularly good at understanding codebases holistically.

Devstral Small 2

Mistral’s coding-focused model.

It scores **68.0% on SWE-bench Verified** as a 24B model under Apache 2.0, and it runs on a single RTX 4090 or a Mac with 32GB RAM. That makes it a strong contender in the **mid tier category.**

I haven’t spent as much time with this one, to be transparent, but Mistral’s track record with efficient models is strong.

Worth evaluating if you’re already in the Mistral ecosystem.

Kimi K2.5

From Moonshot AI, Kimi K2.5 is ambitious: 1.04 trillion total parameters, 32B active, multimodal, and designed for agentic workflows.

Kimi K2.5's flagship feature is **Agent Swarm,** the ability to coordinate up to 100 sub-agents working in parallel, achieving 4.5x execution speedup. This is a genuinely differentiating feature worth calling out.

The multimodal aspect is also interesting for coding, think screenshot-to-code, UI debugging from visual input, or diagram interpretation.

The 32B active parameter count means it needs decent hardware, but the total-to-active ratio is the most extreme on this list.

Setting Up Local Models

Enough theory. Let me show you how I actually run this stuff.

Setup 1: Ollama + Claude Code with Local Models

This is my daily driver setup.

Step 1: Install and pull your model

```
# Install Ollama (if you haven't)
curl -fsSL https://ollama.ai/install.sh | sh

# Pull a model - I recommend starting with Qwen3.5-2
ollama pull qwen3.5:27b

# Or if you're on constrained hardware, the MoE vari
ollama pull qwen3.5:35b-a3b-q4_K_M
```

Step 2: Configure Claude Code to use your local model

```
# Set these environment variables (add to your .bash
export ANTHROPIC_BASE_URL="http://localhost:11434"
export ANTHROPIC_AUTH_TOKEN="ollama"

# Now just run Claude Code as normal
claude
```

That's it. Claude Code's agentic loop, i.e. file reading, editing, terminal commands, the whole thing works against your local model.

The `ANTHROPIC_AUTH_TOKEN` can be any non-empty string since Ollama doesn't require authentication.

Make sure Ollama is actually running and has finished loading the model before launching Claude Code. I spent an embarrassing twenty minutes debugging connection errors before realizing I hadn't started the Ollama server. Run `ollama serve` in a separate terminal if it's not running as a system service.

Note: Multiple users reported issues with this approach, specifically with Qwen3.5–35B-A3B getting stuck in infinite loops.

You also have the following options:

- Using llama.cpp directly instead of Ollama (multiple users report better results)
- Disabling extended thinking mode
- Using LiteLLM as a proxy for better timeout control

Setup 2: OpenAI Codex CLI with Local Models

OpenAI's Codex CLI also supports local models via its `--oss`

flag:

```
# Install Codex CLI
npm install -g @openai/codex

# Run with local Ollama model
codex --oss

# Or specify the model explicitly
OPENAI_BASE_URL="http://localhost:11434/v1" codex --
```

The `--oss` flag is a nice touch, it auto-configures the CLI to look for a local Ollama instance. Less fiddling with environment variables.

Setup 3: Running Qwen3.5 Locally

Let me walk through the complete setup for my recommended starting point:

```
# 1. Make sure Ollama is running
ollama serve

# 2. Pull the model (this downloads ~16GB for the 27
ollama pull qwen3.5:27b

# 3. Test it interactively first
ollama run qwen3.5:27b "Write a Python function to p
```

```
# 4. For Claude Code integration
export ANTHROPIC_BASE_URL="http://localhost:11434"
export ANTHROPIC_AUTH_TOKEN="ollama"
cd your-project-directory
claude "refactor the authentication module to use JW

# 5. For Codex CLI integration
export OPENAI_BASE_URL="http://localhost:11434/v1"
export OPENAI_API_KEY="ollama"
codex "add comprehensive test coverage to src/utils/
```

Other CLI Tools Worth Knowing

OpenCode: This one has quietly accumulated 100K+ GitHub stars, 700 contributors and 2.5M monthly developers. It supports 75+ providers out of the box, including local Ollama. If you want a single tool that works with everything, this is probably it.

Aider: The veteran of the local-model coding tool space. Excellent git integration, good at multi-file edits. Works well with local models via its OpenAI-compatible API support.

Cline and Roo Code: VS Code extensions that support local model backends. If you prefer staying in your editor rather than using CLI tools, these are the way to go. Cline in particular has good support for agentic workflows within the

VS Code environment.

Hardware Tier Guide

OK, real talk. What do you actually need?

Entry Tier (8GB VRAM) for $0-300 (used GPU)

- **Run:** Qwen3.5-35B-A3B Q4
- **Experience:** Surprisingly capable for single-file tasks. Refactoring, test writing, bug fixes. Struggles with large codebase understanding.
- **Hardware examples:** RTX 3060 8GB, RTX 4060, M1/M2 MacBook Air (8GB)

Mid Tier (16-24GB VRAM) for $400-1200

- **Run:** Qwen3.5-27B, GPT-OSS-20B, Qwen3-Coder-30B-A3B
- **Experience:** This is the sweet spot. These models handle multi-file refactors, understand project structure, and generate production-quality code consistently.
- **Hardware examples:** RTX 4090, RTX 5080, M2/M3 Pro MacBook Pro (18–36GB)

High Tier (64GB+ Mac / Multi-GPU) for $2000-5000

- **Run:** GLM-4.7, Qwen3-Coder-480B quantized, Qwen3.5-122B-A10B

- **Experience:** Near-parity with cloud APIs for most tasks. These models are genuinely excellent.
- **Hardware examples:** Mac Studio M2/M3 Ultra (64–192GB), dual RTX 4090/5090

Ultra Tier (128–256GB) — Budget: $5000+

- **Run:** GLM-5 (full), MiniMax M2.5 (full)
- **Experience:** If you're doing this, you probably don't need me to tell you about it.
- **Hardware examples:** Mac Studio M4 Ultra 256GB, multi-H100 workstation

Here's an example setup: Mac Studio with M2 Ultra and 128GB unified memory. It runs GLM-4.7 and Qwen3.5–27B concurrently, and you can swap between them depending on the task. Total cost including the machine is ~$4000 (e.g. refurbished).

Gotchas and Failure Modes

Because nothing is ever as smooth as an article makes it sound, here's my running list of things that will bite you:

- **Context window limitations:** Most local models top out at 32K-128K tokens. Cloud models like Claude offer 200K+. If you're working with very large codebases, you'll hit this wall. Workaround is to use tools that do

smart context retrieval (Aider is good at this) rather than stuffing entire repos into context.

- **First-token latency:** Loading a large model from disk takes time. My GLM-4.7 takes about 45 seconds to load from cold. Keep Ollama running as a persistent service to avoid this.
- **VRAM isn't RAM:** If your model exceeds available VRAM and spills to system RAM, inference speed drops by 5-10x. Check `ollama ps` to make sure your model is fully GPU-resident.
- **Ollama model naming inconsistencies:** Model names on Ollama's registry don't always match the names used in papers or on Hugging Face. Double-check you're pulling the right variant.
- **Agentic loops can go haywire.** Without the guardrails that cloud providers build into their APIs, local models can sometimes get stuck in loops, especially on ambiguous tasks. Keep an eye on token generation and be ready to Ctrl+C. Setting a `max_tokens` limit in your client configuration helps.
- **Updating models is manual.** There's no auto-update mechanism. New model versions drop regularly. Set a reminder to check for updates monthly, or use commands like `ollama pull --update .`

Where This Is All Going

Hardware is getting cheaper fast. Apple's unified memory architecture has been a gift to local model inference.

Mac Mini with 32GB can run models that would have required a $10,000 GPU setup two years ago. As M4 and M5 chips increase memory bandwidth and capacity, the **high tier** setups I described above will become **mid tier** pricing.

You can start with Qwen3.5–27B or Qwen3.5–35B-A3B depending on your hardware. Use Claude Code with Ollama's Anthropic API compatibility. Run it for a week on real work and track down how often you feel limited compared to the cloud API you were using before.

I think you'll be surprised.

The cloud isn't dead.

For teams, for production systems, for the highest-stakes applications, managed APIs still make sense but for individual developer workflows, your local GPU is now a genuine alternative.

And it keeps your code on your machine, costs nothing per token, and works when your internet doesn't.

Bonus Articles

Local LLMs That Can Replace Claude

Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

agentnativedev.medium.com

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation I have been running local models for a...

agentnativedev.medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD is a spec-driven development workflow + prompt/agent harness that tries to prevent context rot by externalizing...

agentnativedev.medium.com

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

agentnativedev.medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Whether you think it's hype or not, people are already trying to run fully autonomous companies on OpenClaw.

agentnativedev.medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

agentnativedev.medium.com

Codex 5.3 vs. Opus 4.6: One-shot

Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

agentnativedev.medium.com